# Deep Learning Lecture: Architecture Design, Universal Approximation and Backpropagation

Refai Arafat Rossi

*Deep Learning*, Chapter 6, Sections 6.4, 6.4.1, 6.5, 6.5.2 and 6.5.4.

October 2026

We will continue with the same network-security problem: classifying network traffic as Benign, DDoS or Port scan. We will not use the XOR example.
The lesson will build one small, fully connected multilayer perceptron (MLP) and calculate its forward pass, loss, gradients and parameter updates by hand. We will use this same example to explain architecture design, the universal approximation theorem, the Jacobian matrix, the chain rule and backpropagation. Goodfellow presents deep feedforward networks as function-approximation models and explains backpropagation as an efficient way to calculate gradients through the network.

## 1 Begin with our classification problem

A network administrator wants to classify traffic based on two numerical features.

| **Input feature** | **Meaning** | **Example value** |
|---|---|---|
| $x_1$ | Normalized packet-rate feature | 2 |
| $x_2$ | Normalized connection-failure feature | 1 |

The input vector is

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

The network must predict one of three mutually exclusive classes:

$$C_1 = \text{Benign}$$
$$C_2 = \text{DDoS}$$
$$C_3 = \text{Port scan}$$

For our training example, the correct class is DDoS. Its one-hot target vector is

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

A one-hot vector assigns 1 to the correct class and 0 to the others. Our goal is to build a network that calculates $P(C_k \mid \mathbf{x}), k = 1, 2, 3$ and learns parameters that give higher probabilities to the correct classes.

# 2 Section 6.4 — Architecture design

## 2.1 What is a neural network architecture?

Architecture describes how a neural network is organized: its layers, the number of units in each layer, the connections between them, and the operations used to transform the input into the output. For our illustrative problem, choose this architecture:

- **Input layer:** 2 features (Packet rate and connection failures)
- **Hidden layer:** 2 ReLU units (Learns combinations of the input features)
- **Output layer:** 3 Softmax units (Benign, DDoS and Port scan probabilities)

We call this a 2–2–3 MLP: two inputs, two hidden units and three output units. It has one hidden layer. The hidden layer uses ReLU, while the output layer uses Softmax because the task has three mutually exclusive classes. This is a deliberately small teaching network. A practical traffic classifier would normally use more features, and its architecture would be selected using validation experiments.

## 2.2 What does a fully connected layer mean?

A fully connected layer means that each unit in the next layer receives input from every unit in the previous layer. In our network:

- Each of the two hidden units receives both input features.
- Each of the three output units receives both hidden activations.

For the first layer,

$$\boxed{\mathbf{z}^{(1)} = \mathbf{x}^T W^{(1)} + \mathbf{b}^{(1)}}$$

and

$$\boxed{\mathbf{h} = \mathrm{ReLU}(\mathbf{z}^{(1)})}$$

For the output layer,

$$\boxed{\mathbf{z}^{(2)} = \mathbf{h}^T W^{(2)} + \mathbf{b}^{(2)}}$$

where the transposes and vector arrangement here follow our column-vector notation for $\mathbf{x}, \mathbf{z}^{(1)}, \mathbf{h}$, and $\mathbf{z}^{(2)}$. Equivalently, the forward pass can be written using row vectors as $\mathbf{x}^T W^{(1)}$ and $\mathbf{h}^T W^{(2)}$. Softmax converts the output logits into the three probabilities:

$$\boxed{p_k = \frac{e^{z_k^{(2)}}}{\sum_{j=1}^{3} e^{z_j^{(2)}}}}$$

The output with the largest probability becomes the predicted class.

## 2.3 How many trainable parameters does this network have?

Architecture design also determines the number of trainable weights and biases.

- **Input-to-hidden layer:** $2 \times 2 = 4$ weights. There are two hidden biases, giving $4 + 2 = 6$ parameters.
- **Hidden-to-output layer:** $2 \times 3 = 6$ weights. There are three output biases, giving $6 + 3 = 9$ parameters.

Thus, the network contains

$$\boxed{6 + 9 = 15 \text{ trainable parameters}}$$

This count matters because architecture influences model capacity, memory usage, computational cost and the amount of training data that may be needed.

### 2.4 How should we choose an architecture?

There is no single architecture that is best for every task. Goodfellow discusses width, depth and connectivity as design decisions for feedforward networks, with validation performance guiding practical choices.

| **Design decision** | **Teaching interpretation** | **Our example** |
|---|---|---|
| Input width | How many features enter the model? | 2 |
| Hidden width | How many units transform the features at each hidden layer? | 2 |
| Depth | How many successive transformations are used? | 1 hidden layer |
| Output width | How many outputs represent the possible classes? | 3 |
| Connectivity | How do units in adjacent layers connect? | Fully connected |

If the model is too small, it may fail to capture important patterns. If it is unnecessarily large, it may become more expensive to train and more susceptible to overfitting. The choice should consider the task's complexity, available data and computational resources.

## 3 Section 6.4.1 — Universal approximation properties and depth

### 3.1 What is the universal approximation theorem?

The universal approximation theorem tells us that a feedforward neural network with a suitable nonlinear hidden-unit activation and enough hidden units can approximate a very broad class of functions to arbitrary accuracy under appropriate conditions. For example, a sufficiently wide network with one hidden layer can approximate any continuous function on a compact input domain to any specified positive approximation tolerance, given a suitable activation function. ReLU networks also have universal approximation results.
In simple language: A neural network does not need a specially designed formula for every pattern. Its weights, biases and nonlinear activation functions can combine to approximate complicated input-output relationships. For our problem, the target function maps traffic features to a class or class-probability distribution:

$$f^* : \mathbf{x} \longrightarrow \begin{bmatrix} P(\text{Benign} \mid \mathbf{x}) \\ P(\text{DDoS} \mid \mathbf{x}) \\ P(\text{Port scan} \mid \mathbf{x}) \end{bmatrix}$$

The network learns an approximation to this function from labeled examples.

### 3.2 Why not use only a linear model?

Without nonlinear hidden activations, stacking linear layers still gives a linear transformation overall. For example, if $\mathbf{h} = W^{(1)T}\mathbf{x} + \mathbf{b}^{(1)}$ and $\mathbf{z} = W^{(2)T}\mathbf{h} + \mathbf{b}^{(2)}$, then substituting the first equation into the second gives another affine function of $\mathbf{x}$. The extra layer alone does not create a nonlinear feature transformation. ReLU changes this:

$$\mathbf{h} = \text{ReLU}\left(W^{(1)T}\mathbf{x} + \mathbf{b}^{(1)}\right)$$

Because ReLU is nonlinear, the resulting network can represent nonlinear relationships between input features and output classes. For example, the distinction between benign traffic and a DDoS attack might depend on combinations of packet rates, connection failures and other traffic characteristics, rather than on any single feature alone.

### 3.3 Why do we need deep networks if one hidden layer is sufficient in theory?

The theorem concerns the network's representational capability. It does not guarantee that a practical network will learn the desired function efficiently. A single hidden layer may require an enormous number of units for some functions. Additional layers can represent certain compositional or hierarchical functions more efficiently, sometimes using far fewer parameters than a shallow network. Goodfellow notes that deeper ReLU networks can represent functions with a number of linear regions that grows rapidly with depth.

- **Shallow network:** One hidden layer can theoretically approximate complex functions, but may need many hidden units.
- **Deep network:** Multiple hidden layers can build successive representations, potentially capturing complex structures more efficiently.

There are three crucial limitations to the theorem:

1. It does not guarantee that a training algorithm will find the right parameters.
2. It does not guarantee good generalization to unseen traffic.
3. It does not say that deeper networks are always better or easier to optimize.

In practice, depth and width are hyperparameters that must be selected and evaluated using data.

## 4 Section 6.5 — Backpropagation

We now understand the architecture and how it can represent a classification function. The next question is: How can the network learn the values of all 15 parameters? The answer is backpropagation combined with an optimization method such as stochastic gradient descent (SGD). Goodfellow makes an important distinction: backpropagation calculates gradients; gradient descent or another optimizer uses those gradients to update the parameters. Backpropagation itself is a general procedure for efficiently evaluating derivatives through a computational graph.

### 4.1 Forward propagation versus backward propagation

- **Forward propagation:** Calculate activations, logits, probabilities and loss by moving from the input layer toward the output layer.
- **Backward propagation:** Start with the derivative of the loss and propagate gradients back through the output and hidden layers to determine how each parameter affects the loss.
- **Parameter update:** Use the calculated gradients to update the weights and biases, then repeat the process.

### 4.2 Our network's forward pass

For clarity, we will use column vectors throughout the mathematical derivation. The input is

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

We choose the following initial weights and biases. These are illustrative values designed for hand calculation, not the result of training on a real network-traffic dataset.
Input-to-hidden weights:

$$W^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 1.0 \end{bmatrix}, \qquad \mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$$

The pre-activation values for the hidden layer are $\mathbf{z}^{(1)} = W^{(1)T}\mathbf{x} + \mathbf{b}^{(1)}$. Substituting the values:

$$\begin{aligned} \mathbf{z}^{(1)} &= \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} \\ &= \begin{bmatrix} 0.5 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} \\ &= \boxed{\begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}} \end{aligned}$$

Both activations are positive. Applying ReLU therefore gives

$$\boxed{\mathbf{h} = \mathrm{ReLU}(\mathbf{z}^{(1)}) = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}}$$

Hidden-to-output weights:

$$W^{(2)} = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 3 & 0 \end{bmatrix}, \qquad \mathbf{b}^{(2)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

The three logits are

$$\begin{aligned} \mathbf{z}^{(2)} &= W^{(2)T}\mathbf{h} + \mathbf{b}^{(2)} \\ &= \begin{bmatrix} 0 & 0 \\ 1 & 3 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \\ &= \boxed{\begin{bmatrix} 0 \\ 2 \\ 1 \end{bmatrix}} \end{aligned}$$

These correspond to Benign, DDoS and Port scan, respectively. Applying Softmax yields

$$\mathbf{p} = \begin{bmatrix} 0.090031 \\ 0.665241 \\ 0.244728 \end{bmatrix}$$

For the true DDoS label $\mathbf{y} = [0, 1, 0]^T$, the cross-entropy loss is

$$\begin{aligned} L &= -\sum_{k=1}^{3} y_k \ln p_k \\ &= -\ln(0.665241) \\ &= \boxed{0.407606} \end{aligned}$$

This is the starting loss that backpropagation will help us reduce.

# 5 Section 6.5.2 — Chain rule of calculus, Jacobian matrix and gradient

## 5.1 The chain rule for a simple calculation

Suppose $u = 2x, \quad L = u^2$. Then $L = (2x)^2$. The chain rule gives

$$\boxed{\frac{dL}{dx} = \frac{dL}{du}\frac{du}{dx}}$$

Since $\frac{dL}{du} = 2u, \quad \frac{du}{dx} = 2$, we obtain $\frac{dL}{dx} = 2u \times 2 = 4u$. For $x = 3, u = 6$, and the derivative is 24. The key idea is that when one variable depends on another through intermediate operations, we multiply the local derivatives along the path. A neural network is a larger composition of operations. Backpropagation applies this same principle systematically to every relevant weight and bias.

## 5.2 What is a gradient?

For a scalar loss $L$ and parameter vector $\boldsymbol{\theta}$, the gradient is

$$\boxed{\nabla_{\boldsymbol{\theta}} L = \begin{bmatrix} \partial L/\partial\theta_1 \\ \partial L/\partial\theta_2 \\ \vdots \\ \partial L/\partial\theta_n \end{bmatrix}}$$

Each entry measures how the loss changes when one parameter changes, while the other parameters are held fixed. In our network, the parameters include all entries of $W^{(1)}, W^{(2)}$ and both bias vectors. The gradient tells us how each of these parameters influences the loss on our training example.

## 5.3 What is a Jacobian matrix?

A gradient describes the derivatives of a scalar function with respect to its inputs. A Jacobian generalizes the idea to a function that maps a vector to another vector. Suppose $\mathbf{y} = f(\mathbf{x})$ where $\mathbf{x} \in \mathbb{R}^m$ and $\mathbf{y} \in \mathbb{R}^n$. The Jacobian is

$$\boxed{J_f(\mathbf{x}) = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_n}{\partial x_1} & \cdots & \frac{\partial y_n}{\partial x_m} \end{bmatrix}}$$

It records how every output component responds to every input component. For a scalar loss $L$ that depends on $\mathbf{y}$, the vector form of the chain rule is

$$\boxed{\nabla_{\mathbf{x}} L = J_f(\mathbf{x})^T \nabla_{\mathbf{y}} L}$$

Here the superscript $T$ denotes the transpose. This is the central mathematical operation behind backpropagation: a gradient is propagated from one operation to its inputs using the local Jacobian.

### 5.4 The Jacobian of Softmax

Recall that Softmax produces $p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$. Its Jacobian has elements

$$\boxed{\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j)}$$

where $\delta_{ij} = 1$ if $i = j$, and 0 otherwise. For our probabilities $\mathbf{p} = [0.090031, 0.665241, 0.244728]^T$, the Softmax Jacobian is approximately

$$J_{\text{softmax}} \approx \begin{bmatrix} 0.081925 & -0.059892 & -0.022033 \\ -0.059892 & 0.222695 & -0.162803 \\ -0.022033 & -0.162803 & 0.184836 \end{bmatrix}$$

Every row describes how one predicted probability changes as the three logits change. This matters because changing one logit can influence more than one class probability: Softmax normalizes all classes together so their probabilities sum to one.

### 5.5 Combine the Softmax Jacobian with cross-entropy

For our one-hot target, the loss is $L = -\sum_i y_i \ln p_i$. Its derivative with respect to the probabilities is $\frac{\partial L}{\partial p_i} = -\frac{y_i}{p_i}$. Applying the chain rule through Softmax gives

$$\frac{\partial L}{\partial z_j} = \sum_i \frac{\partial L}{\partial p_i} \frac{\partial p_i}{\partial z_j}$$

This simplifies to the important result

$$\boxed{\frac{\partial L}{\partial z_j} = p_j - y_j}$$

In vector form,

$$\boxed{\boldsymbol{\delta}^{(2)} = \mathbf{p} - \mathbf{y}}$$

For our DDoS example:

$$\boldsymbol{\delta}^{(2)} = \begin{bmatrix} 0.090031 \\ 0.665241 \\ 0.244728 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

$$= \boxed{\begin{bmatrix} 0.090031 \\ -0.334759 \\ 0.244728 \end{bmatrix}}$$

These are the gradients of the loss with respect to the three output logits. They initiate the backward calculation.

## 6 Section 6.5.4 — Backpropagation in a fully connected MLP

Now we calculate the gradients for every layer in our 2–2–3 network. The important rule is to start at the output, where the loss is known, then move backward toward the input layer.

### 6.1 Calculate the gradients for the output layer

We already calculated the output-logit gradient:

$$\boldsymbol{\delta}^{(2)} = \begin{bmatrix} 0.090031 \\ -0.334759 \\ 0.244728 \end{bmatrix}$$

The output logits are $\mathbf{z}^{(2)} = W^{(2)T}\mathbf{h} + \mathbf{b}^{(2)}$. Therefore, the gradient with respect to the output weights is

$$\boxed{\nabla_{W^{(2)}} L = \mathbf{h}\boldsymbol{\delta}^{(2)T}}$$

Since $\mathbf{h} = [0.5, 0.5]^T$, the gradient is

$$\nabla_{W^{(2)}} L = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \begin{bmatrix} 0.090031 & -0.334759 & 0.244728 \end{bmatrix}$$

Hence,

$$\boxed{\nabla_{W^{(2)}} L \approx \begin{bmatrix} 0.045015 & -0.167380 & 0.122364 \\ 0.045015 & -0.167380 & 0.122364 \end{bmatrix}}$$

Each entry represents the derivative of the loss with respect to the corresponding output weight. The output-bias gradient is

$$\boxed{\nabla_{\mathbf{b}^{(2)}} L = \boldsymbol{\delta}^{(2)}}$$

The bias gradient is the same as the output-logit gradient because each logit depends on its own bias with derivative 1.

### 6.2 Propagate the gradient to the hidden layer

The hidden activations affect the loss through the output weights. The chain rule gives

$$\boxed{\nabla_{\mathbf{h}} L = W^{(2)}\boldsymbol{\delta}^{(2)}}$$

Using our weights $W^{(2)} = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 3 & 0 \end{bmatrix}$, we calculate the two hidden gradients separately. For hidden unit 1:

$$\frac{\partial L}{\partial h_1} = 0(0.090031) + 1(-0.334759) + 2(0.244728) = \boxed{0.154698}$$

For hidden unit 2:

$$\frac{\partial L}{\partial h_2} = 0(0.090031) + 3(-0.334759) + 0(0.244728) = \boxed{-1.004277}$$

Thus,

$$\nabla_{\mathbf{h}} L \approx \begin{bmatrix} 0.154698 \\ -1.004277 \end{bmatrix}$$

The signs differ because the hidden units affect the output classes through different weights.

### 6.3 Propagate through the ReLU activation

Recall the ReLU function: $h_i = \max(0, z_i^{(1)})$. Its derivative, away from zero, is

$$\mathrm{ReLU}'(z_i^{(1)}) = \begin{cases} 0, & z_i^{(1)} < 0 \\ 1, & z_i^{(1)} > 0 \end{cases}$$

Both pre-activations in our example equal 0.5, so both units are active. The ReLU Jacobian is therefore $J_{\mathrm{ReLU}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. The chain rule gives the gradient of the loss with respect to the hidden pre-activations:

$$\boldsymbol{\delta}^{(1)} = \nabla_{\mathbf{h}} L \odot \mathrm{ReLU}'(\mathbf{z}^{(1)})$$

where $\odot$ denotes element-wise multiplication. Since both derivatives are 1,

$$\boxed{\boldsymbol{\delta}^{(1)} = \begin{bmatrix} 0.154698 \\ -1.004277 \end{bmatrix}}$$

If a hidden unit had a negative pre-activation, the corresponding ReLU derivative would be zero, and this gradient would be blocked at that activation for the current input.

### 6.4 Calculate the gradients for the first layer

The first-layer pre-activation is $\mathbf{z}^{(1)} = W^{(1)T}\mathbf{x} + \mathbf{b}^{(1)}$. Therefore,

$$\boxed{\nabla_{W^{(1)}} L = \mathbf{x}\boldsymbol{\delta}^{(1)T}}$$

Substitute the input and hidden gradients:

$$\nabla_{W^{(1)}} L = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \begin{bmatrix} 0.154698 & -1.004277 \end{bmatrix}$$

This produces

$$\boxed{\nabla_{W^{(1)}} L \approx \begin{bmatrix} 0.309396 & -2.008554 \\ 0.154698 & -1.004277 \end{bmatrix}}$$

The first-layer bias gradient is

$$\boxed{\nabla_{\mathbf{b}^{(1)}} L = \boldsymbol{\delta}^{(1)} = \begin{bmatrix} 0.154698 \\ -1.004277 \end{bmatrix}}$$

We now have the gradients for all 15 trainable parameters. This calculation illustrates why backpropagation is efficient: the gradient at each layer reuses the gradient information already computed at the layer above it instead of independently differentiating the entire network for every parameter.

## 7 Update the parameters using SGD

Backpropagation has calculated the gradients. We now use SGD to update the parameters using the update rule $\theta_{\mathrm{new}} = \theta_{\mathrm{old}} - \eta \nabla_\theta L$. Choose a learning rate of $\eta = 0.1$.

### 7.1 Update the first-layer weights and biases

$W^{(1)}_{\text{new}} = W^{(1)}_{\text{old}} - 0.1\nabla_{W^{(1)}} L$. Substituting the values,

$$\boxed{W^{(1)}_{\text{new}} \approx \begin{bmatrix} 0.469060 & -0.299145 \\ -0.515470 & 1.100428 \end{bmatrix}}$$

Similarly,

$$\boxed{\mathbf{b}^{(1)}_{\text{new}} \approx \begin{bmatrix} -0.015470 \\ 0.600428 \end{bmatrix}}$$

Notice that a negative gradient produces an increase in the corresponding parameter, because SGD subtracts the gradient.

### 7.2 Update the output-layer weights and biases

For the second layer, $W^{(2)}_{\text{new}} = W^{(2)}_{\text{old}} - 0.1\nabla_{W^{(2)}} L$. Therefore,

$$\boxed{W^{(2)}_{\text{new}} \approx \begin{bmatrix} -0.004502 & 1.016738 & 1.987764 \\ -0.004502 & 3.016738 & -0.012236 \end{bmatrix}}$$

The output biases become

$$\boxed{\mathbf{b}^{(2)}_{\text{new}} \approx \begin{bmatrix} -0.009003 \\ 0.033476 \\ -0.024473 \end{bmatrix}}$$

All 15 parameters have now been updated.

### 7.3 Recalculate the prediction

We must perform a new forward pass using these updated parameters. The hidden pre-activations become

$$\mathbf{z}^{(1)}_{\text{new}} \approx \begin{bmatrix} 0.407181 \\ 1.102566 \end{bmatrix}$$

Both remain positive, so

$$\mathbf{h}_{\text{new}} \approx \begin{bmatrix} 0.407181 \\ 1.102566 \end{bmatrix}$$

The output logits are now

$$\mathbf{z}^{(2)}_{\text{new}} \approx \begin{bmatrix} -0.015799 \\ 3.773626 \\ 0.771416 \end{bmatrix}$$

Applying Softmax:

$$\boxed{\mathbf{p}_{\text{new}} \approx \begin{bmatrix} 0.021084 \\ 0.932587 \\ 0.046328 \end{bmatrix}}$$

The probability assigned to DDoS has increased from about 66.52% to 93.26%. The new loss for our DDoS example is

$$\begin{aligned} L_{\text{new}} &= -\ln(0.932587) \\ &\approx \boxed{0.069793} \end{aligned}$$

This is an illustrative single-example update with learning rate 0.1. In practice, SGD trains on many samples over repeated iterations or epochs, and individual updates do not always decrease

| State | Cross-entropy loss | Predicted DDoS probability |
|---|---|---|
| Before SGD | 0.407606 | 66.52% |
| After SGD | 0.069793 | 93.26% |

the loss. The numerical example connects all the ideas from Sections 6.4–6.5: architecture defines the computation; forward propagation calculates the prediction; the cost function measures the error; the chain rule and Jacobians calculate gradients; backpropagation propagates those gradients efficiently; and SGD updates the parameters.

# 8 Final picture: Complete network with nodes, edges and calculations

The diagram below shows the initial network used throughout the calculations. The edge labels are the initial weights, the hidden and output nodes display their computed values, and the right-hand panel shows the target and loss. The footer summarizes the backward gradients and the result of one SGD update.

*Primary textbook: Goodfellow, Bengio and Courville, Deep Learning, MIT Press, Chapter 6. Read Sections 6.4–6.5.4 for architecture design, approximation properties, Jacobians and backpropagation.*